Reachability Search over Convex Transition Sets for Multi-Regrasp Manipulation Planning

Julia López Gómez and Steve Tonneau
School of Informatics, University of Edinburgh, United Kingdom
J.Lopez-Gomez@sms.ed.ac.uk, stonneau@ed.ac.uk

***Abstract*—Manipulation planning has to decide where a robot takes hold of an object and where it lets go, how many times, and how to move in between. The number of contacts is not known before planning, and the configurations in which a grasp is possible form thin, nonlinear sets, so planners either enumerate a finite set of grasps, and can then only find the solutions that set contains, or keep the grasps continuous inside a mixed-integer programme, which has to be told how many grasps to look for. We present a planner that expands convex sets of grasping configurations within an A* search. Intersecting a convex decomposition of the collision-free space of robot and object with a convex model of the grasp constraints yields the sets in which the robot can take hold of the object, move it and let go, and the search carries with each set the object configurations it makes reachable. It needs no bound on the number of grasps, builds its graph from linear programs alone, and returns a plan with the fewest grasps its sets allow, or establishes that none exists. A second stage places the motions without a solver and shortens them with one convex program. On a 10-DOF arm turning a cap among obstacles, it returns the fewest grasps for every turn in at most 0.04 s, while GCS on the same sets takes 40–55 times longer and returns an extra grasp on three of five turns, and a sampling-based manipulation planner solves only the single-grasp query. With eight caps (17-DOF), a weighted search solves every query, up to 15 grasps, in at most 1.7 s, where the sampling-based planner solves none.**

***Index Terms*—Manipulation planning, regrasping, reachability, graph search, convex decompositions.**

I. Introduction

A manipulation plan is a sequence of contacts. The robot takes hold of the object, moves it, lets go, and does so again as many times as the task requires. Choosing that sequence is a combinatorial problem whose cost grows exponentially with its length, and the length itself is unknown before planning, since it follows from the goal, from the joint limits of the robot and from the obstacles around the object. The usual way to attack it is to build a graph whose nodes gather the configurations in which the robot can grasp or release the object, and whose edges are the motions between them [1], and then to search that graph by sampling it [2], [3], by solving a mixed-integer programme over it [4], or by fixing the sequence of modes beforehand and optimising within it [5], [6].

Each of these needs a model of reachability between contacts: which grasps can follow the current one, and how far the object can be turned or carried while it is held. Such a model is nonlinear, because it involves the forward kinematics of the arm and collision with the environment, and the configurations that satisfy it occupy thin, lower-dimensional sets. Handling it exactly at the same time as the combinatorics is impractical, so it is normally approximated, either by precomputing a finite set of grasps [7], [8] or by sampling. Convex decompositions of the collision-free configuration space [9]–[12] are the continuous alternative: they cover free space with convex sets, over which Graphs of Convex Sets return trajectories that are globally optimal with respect to the cover [13].

Convex models of this kind have so far been consumed by mixed-integer programmes, which must be given the number of grasps and whose indicator constraints lose their meaning once the sets are thin (Section IV-G). Searches that leave the number of grasps open work instead on a discretisation of the grasps, which both hides solutions and enlarges the search. This is unfortunate, because the properties we want here are the ones A* already has: it is deterministic, it recovers the shortest sequence under an admissible heuristic, and the number of contacts comes out of the search rather than going into it. The missing piece is a reachability model that A* can expand without discretising it.

For footstep planning, CASSR [14] supplies such a model by expanding convex sets of reachable footholds and fitting the footsteps to the chosen sets afterwards. Manipulation does not inherit this directly. The contact sets are not handed over by the environment but have to be constructed from the grasp constraints, and contact displaces the object, so what is reachable depends on the configuration of the object as much as on that of the robot.

We therefore search over sets of grasping configurations instead of individual grasps. Intersecting a convex decomposition of the collision-free space of robot and object with a convex model of the grasp constraints yields the *transition sets*, the places where the robot can take hold of the object, move it and let go. Our A* expands these sets, and carries with each one the object configurations it makes reachable. When a single object coordinate is tied to a single robot joint, as happens for caps, valves and knobs, those configurations form an interval, which the search propagates and compares without approximation. Our contributions are:

- *transition sets*, a continuous model of grasp reachability. They are convex, they are obtained from a convex decomposition of free space and a convex model of the grasp constraints, and we validate them against a colli-

This work was supported by the UKRI AI Centre for Doctoral Training in Dependable and Deployable AI for Robotics (CDT-D2AIR) **[TODO:** grant number].

sion checker where the decomposition is least reliable (Sections III, IV-A–IV-C);
- a *reachability A** that expands them. It takes no bound on the number of grasps, builds its graph from linear programs alone, and returns a plan with the fewest grasps its sets allow or establishes that none exists (Sections IV-D–IV-F);
- a procedure that turns the returned sequence of sets into robot and object configurations, placed without a solver and shortened by one convex program (Section IV-E), and an evaluation against GCS, a mixed-integer formulation and a sampling-based manipulation planner, on a 10-DOF scene with one cap and a 17-DOF scene with eight (Sections V–VI).

II. Related Work

A. Regrasp and manipulation planning

The first regrasp planners searched over finite sets of grasps and placements [15], [16]. The idea persists in regrasp graphs built over thousands of precomputed grasps [7], and in a bimanual planner certified complete with respect to its discretisation [8]. Such planners are complete only with respect to the grasps they enumerate: a solution that needs a grasp between two discretised ones is missed. The manipulation graph [1] made grasps and placements continuous. It was extended to multi-modal planning [2], to asymptotically optimal sampling [17], and to constraint graphs explored with constrained samplers [3], [18]. Their guarantees are asymptotic: they hold in the limit of infinitely many samples. Task and motion planners add symbolic decisions on top [19]. In optimisation-based TAMP, an outer search enumerates skeletons with a fixed number of mode switches, and a local solver handles each one [5]. Closest in spirit, Liu et al. [20] plan contact-rich planar manipulation on a graph of reachable sets of object poses **[CHECK:** characterisation]. Our transition sets are the nodes of the manipulation graph of [1], built as convex sets rather than sampled. This is what lets us guarantee completeness and the minimum number of grasps after a finite computation, relative to the sets.

B. Contact planning over convex sets

Footstep planning faces the same combinatorics as regrasping: a contact sequence whose length is not known in advance. A* over discretised footholds [21] is complete only at the resolution of the discretisation. Mixed-integer programs over convex terrain [4] avoid discretisation and are globally optimal, but the number of steps, or a bound on it, must be fixed, and their solve times are hard to bound. ℓ_1 relaxations [22] make them fast at the cost of these guarantees. **[CHECK:** SL1M characterisation.] CASSR [14] instead searches with A* over convex sets of footholds reachable from the start, expanded by Minkowski sums, and places the footsteps with a quadratic program once the sets are chosen. CASSR discretises the yaw of the feet, and inflates its heuristic for speed, so its optimality is empirical rather than guaranteed. We follow the same principle, but a node's reachable set carries the configuration of the object rather than a foothold. In the coupled tasks we consider it is an interval, which lets us keep an admissible heuristic and exact pruning, with nothing discretised.

Graphs of Convex Sets (GCS) [13], [23] compute shortest paths over convex decompositions through a convex relaxation, and have been extended to contact-rich planar pushing [24]. They plan the motion of a robot in one mode, with no switches of grasp and no object state carried from one switch to the next. A GCS path also visits each set at most once, so a plan that comes back to the same set, as every regrasp does, needs the graph unrolled once per grasp, with the number of grasps bounded in advance (Section VI-B). GCS* [25] searches such graphs heuristically, and is complete and optimal when it can decide exactly whether one partial path dominates another; in general that check must be approximated. With interval reachable sets, it is exact.

C. Convex decompositions of free space

Convex decompositions inner-approximate free space with convex sets. IRIS grows them by alternating separating hyperplanes and inscribed ellipsoids [9], and C-IRIS certifies them collision-free with sums-of-squares programs [10]. IRIS-NP, IRIS-NP2 and IRIS-ZO scale further under a probabilistic guarantee: the fraction of a region's volume in collision is below a threshold, with a given confidence [11], [26]. Clique covers automate the seeding [12], and GPU implementations approach online rates [27]. A volume guarantee says nothing about sets of near-zero volume, such as grasp sets. This is why we validate the transition sets separately (Section IV-C).

III. Problem Formulation

A. Transit, transfer and transition sets

A robot with configuration $x \in \mathcal{X} \subset \mathbb{R}^{n_r}$ (arm and fingers) manipulates an object with configuration α in a known static scene. The state $q = (x, \alpha)$ lives in the composite configuration space $\mathcal{C}$ [1], and $\mathcal{C}_{\text{free}} \subseteq \mathcal{C}$ is the set of states without collisions other than the contacts a grasp requires. The object cannot move on its own. In the *placement space* $\mathcal{C}_P$ it rests stably, and in the *grasp space* $\mathcal{C}_G$ the robot holds it. A manipulation path alternates *transit* segments in $\mathcal{C}_P$, where the robot moves alone and α is constant, and *transfer* segments in $\mathcal{C}_G$, where the robot moves the object at a fixed relative pose. The two can only meet where the object is at once grasped and stably placed, in $\mathcal{C}_G \cap \mathcal{C}_P \cap \mathcal{C}_{\text{free}}$. We call these the *transition sets*; they are the nodes of the manipulation graph [1].

B. Multi-regrasp tasks with a coupled object coordinate

We study the tasks captured by the following assumption, which include caps, knobs, valves and cranks.

Assumption 1: (i) The object has one coordinate $\alpha \in [\alpha_{\min}, \alpha_{\max}]$ and is always stably placed, so $\mathcal{C}_P = \mathcal{C}$. (ii) Holding the object couples α to one robot joint w, the *wrist*: during a transfer, $\dot{\alpha} = \dot{w}$. The two axes may be physically opposed, as they are in our experiments, in which case α is measured in the opposite sense; this is a choice of sign,

not an assumption, and we return to it in Remark 1. (iii) Neither the grasp constraints nor collisions depend on α: $\mathcal{C}_G = G \times [\alpha_{\min}, \alpha_{\max}]$ and $\mathcal{C}_{\text{free}} = \mathcal{X}_{\text{free}} \times [\alpha_{\min}, \alpha_{\max}]$ for some $G, \mathcal{X}_{\text{free}} \subseteq \mathcal{X}$.
Point (iii) holds for rotationally symmetric objects, such as the cap in our experiments. Point (i) has a consequence we rely on throughout: every grasping configuration is also a placement, so a transition set is simply a collision-free piece of the grasp space, $G \cap \mathcal{X}_{\text{free}}$ up to the object range. A transition set is therefore not only where the robot switches between the two kinds of motion, but also where it carries the object, so one set can host a grasp, a whole turn and the release that follows.

The wrist has a limited range $[w_{\min}, w_{\max}]$, so turning the object by more than that range requires several grasps. We fix the positive sense of α and of w so that both increase towards the goal, which costs no generality (Remark 1) and lets us write $\alpha_{\text{init}} \leq \alpha_{\text{goal}}$. Since α affects neither collisions nor grasps, a plan never gains from turning the object backwards, and the task works like a ratchet. A *stroke* grasps the object and drives the wrist forward, raising α by the wrist travel; one stroke is one grasp, so counting strokes counts grasps. The robot then lets go in one of two ways. In a *return* it winds the wrist back with α fixed, ready to begin another stroke in the same set; in a *release* it leaves the set. Everything else is transit in $\mathcal{X}_{\text{free}}$ at fixed α.

Remark 1: Reversing the direction of the task, screwing instead of unscrewing, reverses both coordinates at once: to turn the object the other way the wrist must also advance the other way. The map $(\alpha, w) \mapsto (-\alpha, -w)$ therefore sends one direction of the problem to the other, exchanging w_k^- with w_k^+ and a_{ik}^- with a_{ik}^+. A planner that handles one direction handles both, by applying that reflection when the goal lies below the start.

C. Problem statement

Given $q_{\text{init}} = (x_{\text{init}}, \alpha_{\text{init}})$ and $q_{\text{goal}} = (x_{\text{goal}}, \alpha_{\text{goal}})$, find a continuous path from q_{init} to q_{goal} in $\mathcal{C}_{\text{free}}$, made of transit segments, strokes and returns, that uses the fewest grasps. Ties are broken by the number of changes of convex set along the path; path length is not optimised.

In terms of reachability, let $\mathcal{R}_k \subseteq \mathcal{C}_{\text{free}}$ be the set of states reachable from q_{init} with at most k grasps. The minimum number of grasps is the smallest k such that $q_{\text{goal}} \in \mathcal{R}_k$, and the goal is unreachable if there is no such k. Neither $\mathcal{R}_k$ nor $\mathcal{C}_{\text{free}}$ can be computed exactly. Our planner computes $\mathcal{R}_k$ exactly for an inner approximation of $\mathcal{X}_{\text{free}}$ and G by convex polytopes, and its guarantees hold relative to that approximation.

IV. Method

The planner has an offline stage, which depends only on the robot, the object and the scene, and an online stage that is run for each query. Offline, it builds the free sets (Section IV-A) and the grasp set (Section IV-B), and intersects them to obtain validated transition sets and the graph over them, using linear programs alone (Section IV-C). Online, the reachability A* chooses the sequence of sets and grasps (Section IV-D), and a second stage turns it into motions (Section IV-E). New goals and start configurations reuse the offline stage unchanged. Fig. 1 illustrates the online stage on the task of Section V, which we use throughout as a running example.

A. Free sets

We inner-approximate $\mathcal{C}_{\text{free}}$ by polytopes $P_j = \{q : A_j q \leq b_j\}$, $j = 1, \ldots, J$, computed with clique covers of visibility graphs [12] and IRIS-NP2 regions [26] in Drake [28]. By Assumption 1(iii), the object coordinate does not affect collisions. We therefore replace each region by its slice at $\alpha = 0$, $F_j = \{x : (x, 0) \in P_j\}$. This is sound: if $(x, 0)$ is collision-free, so is (x, α) for every α. The robot can then occupy any $x \in F_j$ at any object angle, and the planner tracks α separately.

B. Grasp set

A grasp fixes the pose of the hand frame E relative to the object frame C, a nonlinear function of the arm joints. We write the resulting conditions as

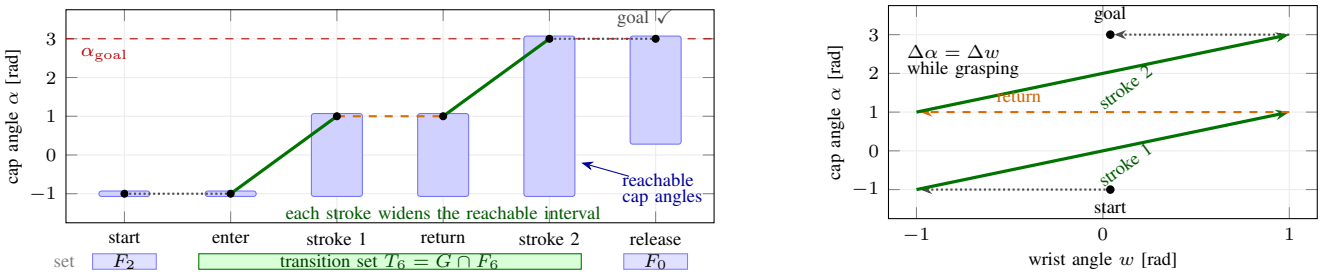
$$h(x) = 0, \qquad g(x) \leq 0, \tag{1}$$

where h collects the equalities a valid grasp must satisfy, typically the components of ${}^C p^E(x)$ and of the hand axes that the grasp pins down, and g the inequalities, such as tolerance bands on the remaining components, an approach direction and a finger opening. Section V instantiates them for our tasks. The set $\{h = 0,\, g \leq 0\}$ is thin and curved, so it cannot be written as a polytope. We approximate it locally, using two ideas from NLP sampling [29]. First, a seed grasp x_0 is found by Gauss–Newton minimisation of the constraint violation $\|[h(x); \max(g(x), 0)]\|^2$ from a random start. Second, the directions that preserve the grasp to first order are the null space of the Jacobian J_h of the equality constraints, which NLP sampling uses to move along constraint manifolds. The grasp set keeps the configurations that stay within ϵ of this null space, around x_0:

$$\begin{aligned} G = \{x : |h_0 + J_h(x - x_0)| \leq \epsilon, \quad g_0 + J_g(x - x_0) \leq \epsilon, \\ x_{\min}^G \leq x \leq x_{\max}^G, \quad |x_j - x_{0,j}| \leq \rho \quad \forall j \in \mathcal{A}\}, \end{aligned} \tag{2}$$

where h_0, J_h, g_0, J_g are evaluated at x_0, ϵ is a tolerance, and $\mathcal{A}$ are the joints other than the wrist.

Two properties shape this set. First, whenever a joint's motion leaves h and g unchanged, its columns in J_h and J_g vanish and G spans that joint's entire range. This is what allows a stroke to run inside a single set, and it is exact rather than approximate: it holds for the wrist of any arm whose last axis passes through the origin of E along the approach direction, since rotating about it moves neither the position nor the approach axis of the hand. Second, J_h leaves a null space of dimension $n_r - \operatorname{rank}(J_h)$ that the constraints of (2) bound only through the joint limits. Far from x_0 the linearisation is meaningless, so the trust region ρ keeps the set where the linearisation is valid; Section VI quantifies what happens



(a) What the search tracks

(b) The same plan in the wrist–cap plane

Fig. 1. The reachability A* on the running example, scene 1cap_2obs of Section V, with transition sets spanning the whole wrist range as on an earlier decomposition (Appendix A): turn the cap from $\alpha = -1$ to 3 rad and bring the arm back to its start. (a) Each column is one node on the returned path: the set the robot is in (strip) and the interval of cap angles reachable there (bar). The black line is the cap angle along the final plan. Entering the transition set T_6 changes nothing; each stroke widens the interval by up to the wrist range of T_6 (2 rad); returns and transit keep it. Leaving T_6 towards F_0 needs some further wrist motion, which raises the foot of the last bar. The search stops at the first node whose set contains the goal arm configuration and whose interval contains α_{goal}. (b) While grasping, the cap turns as the wrist moves forward; returns move the wrist back with the cap fixed. Two strokes are the minimum, since each turns the cap by at most 2 rad. **[TODO:** regenerate once the transition sets are validated (E5).]

without it. Seeds on other parts of the grasp manifold give further grasp sets that the search treats in the same way. We use a single seed in this paper (see Section VII).

C. Transition sets and the graph

For every free set with $F_k \cap G \neq \emptyset$, we form the transition set $T_k = G \cap F_k$. Two LPs give its extreme wrist values, $w_k^+ = \max\{w : x \in T_k\}$ and $w_k^- = \min\{w : x \in T_k\}$, together with points $x_k^+, x_k^- \in T_k$ attaining them. A stroke inside T_k turns the object by at most $L_k = w_k^+ - w_k^-$, and we discard sets with $L_k < L_{\min}$. Since G is thin in every direction but the wrist, T_k is a sliver along the wrist axis, and by convexity the whole segment $[x_k^-, x_k^+]$ lies inside it: the robot can sweep the entire wrist range of T_k without leaving it.

a) Validation: The free sets are collision-free only in a volume sense: IRIS-NP2 bounds the fraction of each region's volume in collision [26]. A transition set has almost no volume, so it can lie entirely in collision without violating that bound. We therefore check the decomposition where the transition sets lie. Because they are slivers along the wrist, this is cheap: we sweep the wrist densely at each seed grasp with a collision checker, and cut every configuration that a region calls free but the checker rejects off that region with a separating plane, without cutting the free part of the sweep (Section V). The regions stay polytopes, and the transition sets follow. This is a dense check, not a certificate.

b) Graph: The graph has two kinds of edges. A *transit* edge joins F_i and F_j if they intersect, and its door point is the Chebyshev centre of $F_i \cap F_j$. A *switch* edge joins F_i and T_k if they intersect. For it, we store the wrist range $[a_{ik}^-, a_{ik}^+]$ of $F_i \cap T_k$ and the points attaining its ends. Each of these quantities is an LP feasibility check, a Chebyshev centre or an extreme point. No integer variables or vertex enumeration are needed, and the number of LPs is $O(J^2 + JK)$ for J free sets and K transition sets.

Algorithm 1 A* search; the parts specific to our problem are in blue

```
1:  addStartNodesToQueue()                          ▷ Sec. IV-D1
2:  while hasNodesToCheck() do
3:      nodeToExpand = getCheapestNode()
4:      if nodeAlreadyExpanded() then               ▷ Sec. IV-D4
5:          skipToNextNodeInQueue()
6:      if hasReachedTheGoal() or hasTimedOut() then
7:          stopSearch()
8:      childNodesToCheck = expandNode()            ▷ Alg. 2
9:      for childNode ∈ childNodesToCheck do
10:         if nodeAlreadyExpanded(childNode) then
11:             continue                            ▷ skip node
12:         costOfEdge = getCostOfEdge()            ▷ Sec. IV-D6
13:         addEdgeToGraph(childNode, costOfEdge)
14:         costToGoal = estimateCostToGoal()       ▷ Eq. (3)
15:         nodeCost = costOfPath(childNode) + costToGoal
16:         addNodeToQueue(childNode, nodeCost)
17: plan = getBestPathToEndNode() ▷ NONE if queue empty
```

D. Reachability A*

We use A* in the same form and with the same syntax as [14], [21] (Algorithm 1). Only the node structure, the expansion, the check for nodes already expanded, the goal test and the costs are specific to our problem. They are highlighted in blue and described below.

1) The A algorithm:* A* keeps a priority queue ordered by the cost of the path to each node plus an estimate of the cost to the goal (line 15). It starts with one node per free set containing x_{init}, with the interval $\{\alpha_{\text{init}}\}$ (line 1). At each iteration, it takes the cheapest node (line 3) and skips it if an equivalent or better node was already expanded (line 4). If the node is a goal (line 6), the plan is read by following parents back to the start (line 17). Otherwise, the node is expanded (line 8), and its children are costed and queued (lines 9–16). If the queue empties first, no plan exists within the sets.

Struct 1 Node

parent: NODE* ▷ parent node
setId: INT ▷ free set F_i or transition set T_k
interval: $[\alpha^-, \alpha^+]$ ▷ reachable object configurations
phase: ENUM ▷ transition sets: entered, holding, released
entryRange: $[a^-, a^+]$ ▷ transition sets: wrist range at entry

Algorithm 2 expandNode, for a node in set S with interval $[\alpha^-, \alpha^+]$, phase ϕ and entry range $[a^-, a^+]$

```
1:  nodeList = []
2:  if S = F_i is a free set then
3:      for all F_j with F_i ∩ F_j ≠ ∅ do                    ▷ transit
4:          nodeList.add(F_j, [α⁻, α⁺])
5:      for all T_k with F_i ∩ T_k ≠ ∅ do                    ▷ enter
6:          nodeList.add(T_k, [α⁻, α⁺], entered, [a⁻_ik, a⁺_ik])
7:  else                                                     ▷ S = T_k is a transition set
8:      if φ ∈ {entered, released} then                      ▷ stroke
9:          c ← w⁺_k − a⁻ if φ = entered, else c ← L_k
10:         β ← min(α⁺ + c, α_goal, α_max)
11:         nodeList.add(T_k, [α⁻, β], holding)
12:     if φ = holding then                                  ▷ return
13:         nodeList.add(T_k, [α⁻, α⁺], released)
14:     if φ ∈ {entered, holding} then                       ▷ release or leave
15:         for all F_j with F_j ∩ T_k ≠ ∅ do
16:             d ← max(0, a⁻_jk − a⁺)                       ▷ forced turning
17:             if φ = holding or d = 0 then
18:                 nodeList.add(F_j, [α⁻ + d, α⁺])
19: return nodeList
```

2) Node structure: As in CASSR, a node does not represent a configuration but a set reachable from the start (Struct 1). A node in a free set F_i with interval $[\alpha^-, \alpha^+]$ states that every $x \in F_i$ is reachable together with every $\alpha \in [\alpha^-, \alpha^+]$; since F_i is convex and α is constant during transit, a straight line joins any two of its points without leaving it. A node in a transition set T_k also records the wrist range $[a^-, a^+]$ through which T_k was entered, and a phase: *entered* (not yet grasping), *holding* (after a stroke) or *released* (after a return, with the wrist at w_k^-). The interval carries all the information about the path so far: reaching F_3 before any grasp and after a stroke of 2 rad gives the intervals $\{-1\}$ and $[-1, 1]$, two different nodes.

3) expandNode(): Algorithm 2 lists the children of a node. A stroke turns the object by any amount up to c, the wrist travel left in T_k ($w_k^+ - a^-$ after entering, L_k after a return), so it raises α^+ and keeps α^-. Since the object can only be grasped inside transition sets, strokes are the only moves that widen the interval. Leaving T_k towards F_j while holding the object is a release, and it can force some turning: the wrist only advances while grasping, so reaching the exit range $[a_{jk}^-, a_{jk}^+]$ from the entry range turns the object by at least d, which raises α^-. Without holding, the robot may only leave if $d = 0$. Children with an empty interval are discarded, and children in the same T_k keep the entry range of their parent. **[CHECK:** d trims only the least-turned end of the interval, and measuring it from a^+ is exact there: the least-turned configuration that can leave is the one that entered as far forward as possible and turned just enough. The open question is physical. A return winds the wrist back without the object, which assumes that letting go frees the wrist; under that same assumption no exit forces any turning, d is always 0, and leaving from *entered* needs no condition. The rule as written only makes sense if the object has to be carried to the doorway, which contradicts the return. Decide which model holds before reimplementing.]

4) nodeAlreadyExpanded(): A node is skipped if an expanded node with the same key has an interval that contains its interval, at no higher cost. The key is the set for free nodes, and the set, phase and entry range for transition nodes. The check is applied when a node is taken from the queue (line 4) and before a child is added to it (line 10). Because the key and the interval describe everything that can still be reached from a node, this check is exact: a skipped node can reach nothing that the expanded one cannot reach at the same or lower cost. In CASSR, two nodes are instead treated as the same if their foothold sets lie within an empirical 2 cm of each other [14]. Unlike CASSR, we also allow the search to return to a set it has already left, since the same set can be revisited with a different interval, and dominance removes the revisits that cannot help.

5) hasReachedTheGoal(): A node is a goal if it lies in a free set F_i with $x_{\text{goal}} \in F_i$ and $\alpha_{\text{goal}} \in [\alpha^-, \alpha^+]$.

6) Cost computation: getCostOfEdge() returns 1 for a stroke, a return or a release, and $\mu \ll 1$ ($\mu = 10^{-3}$) for every other move. Every stroke is followed by exactly one return or release, so the cost of a plan is twice its number of grasps, plus μ times its number of set changes. As long as a plan makes fewer than $1/\mu$ set changes, minimising this cost minimises the number of grasps first and the set changes second. Releasing must cost 1, or leaving and re-entering a transition set would be a cheaper substitute for a return. **[CHECK:** the code also allows a return from the *entered* phase at cost 1, before any grasp; such a move should cost μ for the cost to be exactly twice the number of grasps.] estimateCostToGoal() returns, with $L_{\max} = \max_k L_k$,

$$h(\nu) = 2\left\lceil \frac{\alpha_{\text{goal}} - \alpha^+}{L_{\max}} \right\rceil + \mathbf{1}[\phi = \text{holding}]. \tag{3}$$

It is admissible: each remaining stroke turns the object by at most $L_{\max}$ and is followed by a return or the final release, and a node that is holding the object must still release it. It is also consistent, since no move lowers h by more than its cost. CASSR instead scales its distance-based heuristic, which makes it inadmissible and turns the search into a weighted A* [14]. With an admissible heuristic and exact dominance, the first goal node expanded has the minimum cost.

E. From sets to motions

The search fixes the sets and the grasp actions, e.g., *enter, stroke, return, stroke, release*. As in CASSR, a second stage computes the configurations, in two steps.

Placement. Without a solver, a backward pass from α_{goal} decides how much each visit to a transition set turns the object; it ends exactly at α_{init} because the search proved α_{goal} reachable. Waypoints are placed at door points for transit, in $F_i \cap T_k$ for entering and leaving, and on $[x_k^-, x_k^+]$ for strokes and returns. Each motion is a straight line between two waypoints of one set, so by convexity it lies in that set.

Shortening. The placement is valid but long, since door points and extreme wrist points are far apart. One second-order cone program then moves the waypoints to minimise $\sum_i \|q_{i+1} - q_i\|$ over the same sets, which is the convex restriction of GCS [13] to the sequence the search chose. It keeps the start and the goal, both ends of every motion in that motion's set, and every motion's kind: a stroke keeps $\Delta\alpha = -\Delta w$ with $\Delta w \geq 0$, other motions in a transition set keep $\Delta w \leq 0$, and no other motion turns the object. Each placement is a feasible point of this program, so it is always feasible; if the solver fails, the placement is kept. We check every motion of the result for containment and the coupling.

F. Properties

Proposition 1 (informal): Relative to the free sets $\{F_j\}$ and the transition sets $\{T_k\}$: (i) every motion returned is a straight line inside one set that satisfies the coupling of Assumption 1, so the path is collision-free whenever the sets are; (ii) the search terminates. If a plan exists whose transit motions lie in the free sets and whose strokes lie in the transition sets, it returns one with the minimum number of grasps, with ties broken by the number of set changes. Otherwise, it returns NONE; (iii) the second stage succeeds on every sequence the search returns.

Sketch. (i) follows from convexity of the recorded sets and from the coupling check applied to every motion. (ii) Every interval end the search produces is built from the finitely many wrist values the LPs of Section IV-C return, clipped at α_{goal}, so only finitely many nodes exist per key and the search terminates. With a consistent heuristic and exact dominance, A* expands a goal node of minimum cost first [30]. Entering at the lowest and leaving at the highest wrist value of a switch edge maximises the travel available, so restricting entries and exits to these points loses no plan. **[CHECK:** the two choices are made by different paths, one realising each end of the interval, which is what a reachable set allows; say so explicitly.] (iii) The backward pass mirrors the interval updates of Algorithm 2. **[CHECK:** write the full proof; (iii) is asserted in our implementation but not yet proven. Overlapping transition sets would need chain-aware placement, so each set is its own segment for now.] □

G. Relation to a mixed-integer formulation

The same problem can be posed as a mixed-integer program, in the way convex decompositions are normally used for contact planning [4]: for a fixed number of grasps ℓ, choose keyframes $q_{\text{init}}, x_1, \ldots, x_{2\ell}, q_{\text{goal}}$, let a binary z_{ij} indicate that keyframe i lies in set j, and increase ℓ until the program is feasible. Membership is an indicator constraint, usually in big-M form, $A_j x_i \leq b_j + M(1 - z_{ij})$. Two properties of that formulation are at odds with the sets of Section IV-C. Solvers certify a binary as integral only up to a tolerance τ, so a variable reported as $z_{ij} = 1$ may be $1 - \tau$ and relax every row of its set by $M\tau$; membership is meaningful only for sets much thicker than that, whereas transition sets are deliberately thin. And the continuous relaxation weakens as M grows, so the search tree grows with it. Tight per-row constants and extended formulations that model the union by its convex hull both help [31], at the price of a larger model, but neither removes the outer loop over ℓ. The reachability A* has no relaxation and no integrality tolerance, and the number of grasps is an outcome of the search rather than an input.

V. Experimental Evaluation

Our experiments ask four questions. Does the search return plans with the minimum number of grasps its sets allow, for any goal, from a single offline stage? How does it compare with GCS on the same sets and with a sampling-based manipulation planner, counting both offline and online cost? How does it scale with the number of objects? And why must the decomposition be checked where the transition sets lie?

A. Scenes and implementation

A Franka Panda [32] turns caps about fixed vertical axes. Its wrist is restricted to $[-1, 1]$ rad, so any turn larger than the wrist range left free by the scene needs several grasps. Each scene has a floor just under the caps and the robot base.

- **1cap_2obs** (10-DOF): seven arm joints, two fingers and one cap, 0.5 m in front of the base, with two cylindrical obstacles (radius 5 mm) beside it. The obstacles block part of the wrist range at the grasp.
- **8caps_3obs** (17-DOF): eight caps on an arc of radius 0.5 m around the base, from $-120°$ to $120°$, and three radial walls (0.35 and 0.5 m high) between groups of caps, so that moving between groups means going over or around a wall.

The comparison with the MIQP also uses a 3-DOF task (wrist, finger and cap of a locked arm, 15 regions, a 2π turn). **[CHECK:** 3-DOF task from the MInf report (WSG gripper) or the locked Franka?]

a) Decomposition: Every scene is decomposed by one procedure. A cap's collision geometry is a cylinder about its own joint axis, so no collision depends on a cap angle: we compute $\mathcal{C}_{\text{free}}$ in the 9-D space of arm and fingers with the caps welded, which is exactly the slice of Section IV-A. The procedure has four steps. (1) For each cap, a seed grasp is refined onto (1) by an NLP. (2) IRIS-NP2 [26] grows regions along each grasp's collision-free wrist runs, from an ellipsoid that is long along the wrist and thin across it; a round initial ellipsoid gave regions only about 0.2 rad long in the wrist. (3) A clique cover [12] with these regions as initial sets runs to a 0.98 coverage target. (4) Points that a region calls free but the collision checker rejects are cut off by separating planes, never cutting the free part of a grasp's wrist sweep: on dense

wrist sweeps at each grasp, around the caps and uniformly over the joint box. Table I reports the result.

b) Grasp constraints: The constraints (1) ask the hand to be centred over the cap, within a height band, with its approach axis $z_E = {}^{C}R^{E}(x)e_z$ pointing down:

$$h(x) = [\, p_x,\ p_y,\ z_{E,x},\ z_{E,y}\,]^\top, \tag{4}$$

$$g(x) = [\, p_z - z_{\max},\ z_{\min} - p_z,\ z_{E,z} + c\,]^\top, \tag{5}$$

with $p = {}^{C}p^{E}(x)$, $[z_{\min}, z_{\max}] = [0.105, 0.110]$ m and $c = 0.9$, and the fingers held within a 1 mm band around the width of the cap. Here J_h has rank 4, leaving a six-dimensional null space. The grasp set of (2) uses $\epsilon = 10^{-3}$, $\rho = 0.05$ rad and $L_{\min} = 0.05$ rad. The wrist of the Panda satisfies the invariance of Section IV-B exactly: sweeping it over its whole range at a grasp leaves h and g unchanged to machine precision. With several caps, each cap has its own grasp set, and a node carries one interval of α per cap; a move inside a transition set of cap k changes only the interval of α_k, and the heuristic (3) becomes a sum over caps.

c) Implementation: The planner is in Python, with Drake [28] (v1.48). The graph needs tens of thousands of small LPs (emptiness checks, Chebyshev centres, extreme points). We solve each as the regularised QP $\min c^\top x + \frac{\varepsilon}{2}\|x\|^2$ subject to $Ax \le b$, with $\varepsilon = 10^{-8}$, using the dual active-set method of Goldfarb and Idnani [33], which reports infeasibility, and hence emptiness, directly. With MOSEK, every one of these solves checked a licence out, which made the graph of 8caps_3obs take 15 min. MOSEK [34] solves the decomposition's programs, GCS and the second stage, with one licence held throughout. All times are on one AMD Ryzen 9 9950X (16 cores, 32 threads, 60 GB).

B. Baselines and protocol

a) GCS on the same sets: The free sets F_j and the transition sets T_k, with α free within its limits, are the vertices, and the transit and switch edges of Section IV-C are the edges. Each visited vertex holds one straight segment $r_0 \to r_1$, with both ends in its set. The coupling becomes linear constraints on these two points: $r_1^\alpha = r_0^\alpha$ in every F_j, and in every T_k

$$r_1^\alpha - r_1^w = r_0^\alpha - r_0^w, \qquad r_1^w \ge r_0^w,$$

which is a forward stroke. A GCS path visits each vertex at most once, but a plan can need several strokes in the same T_k, with returns in between. So the graph is copied once per grasp:

$$\text{approach} \to \text{grasp}_1 \to \text{between}_1 \to \cdots \to \text{grasp}_N \to \text{depart},$$

with $N = \lceil D/L_{\max} \rceil + 1$ for a turn D. Every grasp_n also connects to the depart layer, and the approach layer also connects directly to the goal. The cost is the path length, plus 10 per grasp, plus μ per other change of set; ten exceeds any path length here, so the fewest grasps come first, as in the search. The between layers hold the free sets that meet a transition set; the approach and depart layers hold those, the free sets containing x_{init} or x_{goal}, and the free sets of the A*'s plan. We solve it with Drake's `SolvePath`: a convex relaxation, rounded to at most 10 candidate paths, each finished by a convex restriction.

b) Manipulation RRT: As the sampling-based baseline we use the Manipulation RRT of HPP [3], which plans on the manipulation graph [1], built by HPP's constraint-graph factory on the same robot and scene (exported from Drake, with Drake's collision geometry and filters). The hand grasps the cap's handle 0.1075 m above it, pointing down, at any yaw; outside a grasp the cap joint is locked at its current angle; and the fingers are locked at the grasp width. Motions are validated continuously (HPP's `Dichotomy`, with a progressive projector in constrained states): HPP's default discretised check returned plans that Drake rejects. Paths are not shortcut.

c) Protocol: Every returned motion is re-checked with Drake: collisions, the coupling (the cap moves only with the grasping wrist) and the goal. On 1cap_2obs, every method shares one start and one goal, saved with the sets, and the cap turns by $D \in \{0.5, 1.5, 2.4, 3.2, 6\}$ rad. HPP runs 10 seeds per query with a 60 s limit. On 8caps_3obs, for each number K of caps to turn, we draw 20 random queries: K random caps, each turned by a random amount in $[0.5, \pi]$ rad, summing to $2.5K$ rad, with random start and goal arm configurations in the free sets. The A* runs exactly and weighted ($w = 1.5$), with a 60 s limit. HPP runs 5 of those queries per K, 3 seeds each, 60 s each. Online times count each method's work per query: for the A* the search and the second stage; for GCS building its layered graph and solving it; for HPP its `solve()`. The offline stage is reported separately (Table I), and is shared by the A* and GCS. Memory is the peak extra resident memory; each full GCS solve runs in a fresh process.

VI. Results

A. Offline stage

Table I reports the offline stage of both scenes. The decomposition dominates it: 2.8 and 18.6 min, against 0.09 and 2.0 s for the graph. Both are done once per scene, and every query reuses them. On 20,000 uniform samples, the regions wrongly call 7 and 10 configurations free (0.04 and 0.05%), and they cover 91 and 81% of the free samples. On the dense wrist sweeps at the grasps, where the transition sets lie, they call no colliding configuration free and cover 225 of the 230 and 3153 of the 3159 free ones. On 1cap_2obs the obstacles leave the wrist free over at most $L_{\max} = 0.95$ rad, so a turn D needs at least $\lceil D/0.95 \rceil$ grasps.

B. One cap: the A*, GCS and the Manipulation RRT

Table II compares the three methods on 1cap_2obs.

- *The A* always returns the fewest grasps*, $\lceil D/L_{\max} \rceil$, in 0.010–0.037 s, second stage included. It expands 112–398 nodes, in 0.007–0.027 s.
- *Full GCS returns shorter paths, but not always the fewest grasps.* It is 40–55× slower (0.56–1.8 s) and uses 0.8–1.4 GB. At the two turns where it returns as many grasps as the A*, its path is 10 and 19% shorter. At the other

TABLE I

OFFLINE STAGE, DONE ONCE PER SCENE AND SHARED BY THE A* AND GCS. FALSE POSITIVES: CONFIGURATIONS THAT A REGION CALLS FREE AND THE COLLISION CHECKER REJECTS, ON 20,000 UNIFORM SAMPLES AND ON THE DENSE WRIST SWEEPS AT THE GRASPS. COVERAGE: FRACTION OF THE FREE UNIFORM SAMPLES INSIDE A REGION.

	1cap_2obs	8caps_3obs
decomposition time (32 threads)	2.8 min	18.6 min
free sets (seeded along the wrist)	61 (3)	352 (22)
false positives: uniform / sweeps	7 / 0	10 / 0
coverage of the free samples	0.91	0.81
transition sets	4	11
$L_{\max}$ [rad] (per cap)	0.95	1.65–2.0
transit / switch edges	1714 / 10	23,729 / 17
graph time (LPs)	0.09 s	2.0 s

TABLE II

PLANS ON 1CAP_2OBS FROM ONE START AND GOAL, FOR A TURN D. LOWER BOUND: $\lceil D/L_{\max} \rceil$ GRASPS. LENGTH: EUCLIDEAN, IN (x, α). TIME: ONLINE, PER QUERY; FOR GCS ON THE A* SEQUENCE IT INCLUDES THE A* SEARCH AND BUILDING THE LAYERED GRAPH. HPP: SOLVED/RUNS, MEDIAN OVER SOLVED RUNS. EVERY RETURNED PLAN PASSES THE CHECKS OF THE PROTOCOL.

D [rad]	Method	Grasps	Length	Time [s]	Mem. [GB]
0.5	Reach. A* (ours)	**1**	14.10	**0.010**	<**0.01**
	GCS on A* seq.	**1**	14.10	0.019	<**0.01**
	Full GCS	**1**	**11.36**	0.56	0.81
	Manip. RRT, 7/10	**1**	16.67	0.040	<0.1
1.5	Reach. A* (ours)	**2**	16.03	**0.017**	<**0.01**
	GCS on A* seq.	**2**	16.03	0.028	<**0.01**
	Full GCS	3	13.99	0.73	0.92
	Manip. RRT, 0/10	no plan within 60 s			
2.4	Reach. A* (ours)	**3**	18.21	**0.022**	<**0.01**
	GCS on A* seq.	**3**	18.21	0.035	<**0.01**
	Full GCS	4	15.52	0.93	0.78
3.2	Reach. A* (ours)	**4**	20.14	**0.019**	<**0.01**
	GCS on A* seq.	**4**	20.14	0.033	<**0.01**
	Full GCS	**4**	**18.17**	1.04	1.09
6	Reach. A* (ours)	**7**	26.90	**0.037**	<**0.01**
	GCS on A* seq.	**7**	26.90	0.059	<**0.01**
	Full GCS	8	24.22	1.82	1.35
	Manip. RRT, 0/10	no plan within 60 s			

three it returns one grasp more, which its own cost ranks lower: at $D = 1.5$ the A*'s plan is a path of the same GCS graph with cost $16.03 + 2 \cdot 10 = 36.0$, against 44.0 for the plan returned. Rounding the relaxation carries no optimality guarantee, and here it missed the plan with the fewest grasps.

- *GCS on the A* sequence* solves only the convex restriction of the same layered GCS graph to the A*'s sets, in 0.019–0.059 s, A* search included. It returns the A*'s lengths to seven digits: the A*'s second stage (Section IV-E) is that same convex program, so it is optimal for the sets the A* chose. The length gap to full GCS at equal grasps therefore comes from the choice of sets, which the A* makes by the number of set changes, not by length.
- *The Manipulation RRT solves only the one-grasp query*, in 7 of 10 runs, in 0.04 s (median), with paths 18% longer than the A*'s. With two or more grasps it found no plan in any run, neither within 60 s nor, on the scene without the floor, within 600 s. Each regrasp needs the tree to reach a thin grasp set, turn within it and leave it, and a long chain of such steps is unlikely to be sampled.

Counting offline and online cost, the A* and GCS share the 2.8 min decomposition and the 0.09 s graph; GCS then pays 0.6–1.8 s and about 1 GB per query, the A* at most 0.04 s. The Manipulation RRT has no offline stage, but does not solve the multi-grasp queries.

a) Where GCS spends its time: `SolvePath` does not solve the mixed-integer program of GCS [23]: it solves its convex relaxation and rounds it. The relaxation is one conic program in which every edge carries a copy of the segment variables of both its ends, so its size follows the number of edges, which grows with the number of grasps: from 29 vertices and 130 edges at $D = 0.5$ to 77 and 346 at $D = 6$, with time and memory growing with them. The A* solves no program in its search: it does interval arithmetic on extreme points computed offline. On an earlier decomposition of the same scene, with 8 transition sets and 543 edges for one grasp, the same GCS formulation took 44 and 93 s for one and two grasps and did not fit in 60 GB at eight (Appendix A). The cost of GCS thus depends on the number of transition sets, which grows with the number of objects.

b) Why GCS needs the graph unrolled: In GCS a path is a sequence of distinct vertices, and each vertex holds a single segment [23]. A regrasp plan, however, comes back to the same transition set once per grasp, and to the free sets where it lets go and turns the wrist back. GCS can represent such a plan only after the graph is copied by hand, with the coupling written on each copy and the number of grasps bounded before solving: on a two-variable toy ((w, α) in a box, a 4 rad turn, strokes of at most 2 rad), a single copy of the grasp vertex is infeasible, both relaxed and as the exact mixed-integer program. A node of the reachability A* pairs a set with an interval of α, so coming back to a set with a different α is simply a new node, the number of grasps is an output of the search, and the graph is built once for all queries. **[TODO:** Full GCS with returns inside copies of the transition sets (the A*'s model), and on 8caps_3obs.]

C. Eight caps: scaling with the number of objects

Table III reports 8caps_3obs. Of the 160 queries, 15 start or end outside the current free sets (they were drawn for an earlier decomposition, and are kept so that HPP sees the same queries); we leave them out.

- *Weighted A* solves every query*, 145 of 145, with time growing about linearly in K: from 0.13 s for one cap to 1.6 s for eight (15 grasps), all plans certified.
- *Exact A* proves the minimum, but only for few caps.* It solves every query up to $K = 3$, 15 of 20 at $K = 4$ and none at $K = 5$ within 60 s. With K caps, a set can be reached with many combinations of cap intervals that

TABLE III

REACHABILITY A* ON 8CAPS_3OBS AS THE NUMBER K OF CAPS TO TURN GROWS: SOLVED / QUERIES WITH START AND GOAL IN THE FREE SETS, MEDIAN GRASPS AND NODES EXPANDED, AND TIME PER QUERY IN MS, MEDIAN (25TH–75TH PERCENTILE) OVER SOLVED QUERIES. EXACT A* STOPS AT 60 S, AND IS NOT RUN PAST THE FIRST K WHERE IT SOLVES NOTHING. EVERY PLAN PASSES THE CHECKS OF THE PROTOCOL.

	Weighted A* ($w = 1.5$)				Exact A*	
K	Solved	Grasps	Nodes	Time [ms]	Solved	Time [ms]
1	17/17	2	396	129 (110–168)	17/17	227 (136–301)
2	19/19	4	751	295 (283–312)	19/19	1057 (832–1268)
3	18/18	6	1102	455 (393–480)	18/18	5080 (4399–6385)
4	20/20	7	1438	597 (573–647)	15/20	37954 (20906–43164)
5	19/19	9	1795	803 (757–835)	0/19	>60 000
6	20/20	11	2133	1057 (1007–1100)	–	–
7	17/17	13	2525	1328 (1258–1350)	–	–
8	15/15	15	2891	1575 (1555–1602)	–	–

TABLE IV

CHOOSING GRASPS WITH THE MIQP AND WITH THE REACHABILITY A*

Task	Method	Grasps	Time	Outcome
3-DOF	MIQP	11	[TBD]	solved; GCS connected
	Reach. A*	[TBD]	[TBD]	[TBD]
1cap_2obs	MIQP	–	[TBD]	numerical failure [CHECK:]
	Reach. A*	2	0.017 s	solved ($D = 1.5$)

do not dominate one another, and the heuristic, a sum of per-cap bounds, ignores the travel between caps.

- *Weighting cost nothing here.* On all 69 queries both solved, the weighted search returned exactly as many actions as the exact one. Its bound guarantees at most 1.5 times the minimum cost.
- *The Manipulation RRT found no plan* in any of its 120 runs (5 queries and 3 seeds for each K), already at $K = 1$, where a query turns one cap by 2.5 rad and needs at least two grasps.

D. Comparison with the MIQP

Table IV compares the two ways of choosing grasps. On the 3-DOF task, the MIQP returns 11 grasps after proving 10 infeasible. Each stroke turns the cap by 0.604 rad, the wrist range of the grasp set in that decomposition, and $\lceil 2\pi/0.604 \rceil = 11$. GCS then connects all 23 transit and transfer segments. On 1cap_2obs it does not return a reliable solution. The numbers make the argument of Section IV-G concrete: the grasp set is $2\epsilon = 2 \times 10^{-3}$ thick, while $M = 10^6$ and MOSEK's default integrality tolerance $\tau = 10^{-5}$ [34] give $M\tau = 10$ rad of slack on a set reported as occupied, more than any joint range. **[CHECK:** MIQP outcome and solve time on 1cap_2obs with the current decomposition; run the reachability A* on the 3-DOF task and check that it also returns 11.]

E. Why the decomposition must be checked at the grasps

A planner whose guarantees are relative to its sets inherits their errors. Region generators bound the fraction of a region's volume in collision, and transition sets have almost no volume (Section IV-C), so that bound says nothing about them. On an earlier decomposition of 1cap_2obs, generated by a clique cover alone, we swept the wrist over $[-1, 1]$ at the seed grasp, at 99 points. The collision checker found it free only on $[-1, -0.19]$ and $[0.70, 0.92]$ rad, but all four decompositions we had computed (IRIS-NP2 with 42 and 46 regions, two IRIS-ZO covers with 64) called all 99 points free. A two-grasp plan for a 4 rad turn was therefore valid relative to the sets, but collided; with the true free interval of 0.81 rad the minimum was five grasps, a different discrete structure, not just a different geometry. This is why the decomposition seeds regions along the free wrist runs at each grasp and cuts the false positives found on dense wrist sweeps (steps 2 and 4). On the current decompositions those sweeps find no false positive (Table I), and every plan of Tables II–III passes the collision checker. A dense check is not a certificate: certifying the thin transition sets with C-IRIS [10] would be.

F. The trust region on the grasp set

Without the trust region ρ, the extreme points that the graph construction returns lie far outside the region where the linearisation holds: on 1cap_2obs they lay 2.27 rad from the seed in one arm joint, with $\|h\|_\infty = 0.95$ and the hand 0.80 m away from the cap. With $\rho = 0.05$ rad, $\|h\|_\infty$ stays near 2×10^{-3} along the plans we report **[CHECK:** numbers from a current run].

VII. DISCUSSION AND LIMITATIONS

a) Validating where it matters: The weakest link is not the search but the sets it searches over (Section VI-E), and a dense check is not a certificate. Seeding the decomposition along the grasps' wrist runs and cutting the false positives found there removed every error we measured, but only at the sampled points. Certifying the thin transition sets with C-IRIS [10] would strengthen the guarantees without changing the search.

b) Constructing grasp sets: We take two ideas from NLP sampling [29], its Gauss–Newton phase and its null-space view of the constraints, but not its interior samplers, which we first tried and could not turn into usable polytopes. Seeding IRIS regions with grasp samples produced many tiny polytopes, because a region grown around a thin, curved set is bounded by its curvature in every direction. Fitting a union of polytopes to the samples with a small network, each neuron a half-space, a region the maximum over its half-spaces and the set the minimum over regions, never fitted the grasp set closely enough to plan with. Hit-and-run itself mixed poorly in the wrist and cap coordinates, since its line search shares one step size across all of them. What works instead is to exploit the structure of the constraints, above all their exact invariance to the wrist, rather than to rediscover it from samples. The price is locality: one seed yields one grasp set, valid near that seed. The restarts of NLP sampling spread seeds across the components of the grasp manifold, and are the natural way to obtain the several grasp sets the search already accepts. **[TODO:** numbers for the attempts, if space allows.]

c) Scope: Exact intervals rely on Assumption 1, which covers caps, knobs, valves and cranks but not general pick-and-place. With more object coordinates, a node's reachable object configurations become sets, which could be propagated with Minkowski sums as in CASSR [14], at the cost of exact dominance.

d) Optimality and offline cost: The search minimises the number of grasps, then the number of set changes, which is only a weak proxy for distance travelled. The second stage minimises length, but over the sets the search fixed, as in CASSR, so path length is not optimal: at equal grasps, full GCS found paths 10–19% shorter (Section VI-B). Adding length to the cost while keeping the search exact needs a position per node, e.g. the door point a node was entered through, with $\lambda\|p - p_{\text{goal}}\|$ added to the heuristic and λ small enough that length never outweighs a grasp; the price is more nodes. A new obstacle layout requires a new decomposition, which dominates the offline cost: 2.8 and 18.6 min on 32 threads for our two scenes, against 0.09 and 2.0 s for the graph. GPU implementations [27] reduce it, and new start and goal queries reuse the graph unchanged.

VIII. Conclusion

A manipulation plan is a sequence of contacts, and separating the choice of contacts from the motions that realise them is what costs feasibility and optimality. We kept the two together by building the manipulation graph out of convex sets and searching over reachability: a node pairs a convex set with the interval of object configurations reachable inside it. For objects with one coordinate coupled to the wrist, the search is complete and returns the fewest grasps the sets allow, without fixing that number in advance and without integer variables, and a second stage places the motions without a solver and shortens them with one convex program. This carries over to regrasping an idea matured in footstep planning, and extends it to contact sets that must be constructed and to contacts that move the object. The guarantees are only as good as the transition sets are faithful near contact, which is why we validate them. On a 10-DOF scene the search returned the fewest grasps in at most 0.04 s, where GCS on the same sets took 40–55 times longer and sometimes an extra grasp, and a sampling-based planner solved only single-grasp queries. With eight caps, a weighted search planned up to 15 grasps in under 1.7 s; proving the minimum exactly remains limited to a few caps.

Appendix

Table V lists every run of the GCS comparison on an earlier decomposition of 1cap_2obs, including the runs that failed or ran out of memory. That decomposition had 42 IRIS-NP2 regions from a clique cover alone, in the 10-D space with the cap jointed and no floor. Its transition sets were cut by hand to the wrist intervals that the collision checker accepts (Section VI-E), so $L_{\max} = 0.81$ rad, and 8 of them were left. All LPs used MOSEK. Section VI-B reports the current decomposition, generated as in Section V.

- *Start and goal.* Each run draws its own start and goal from a randomly seeded grasp, so rows can be compared within a run but not across runs.
- *Run A* saved its start, goal and sets, and reproduces exactly. Its start and goal are the ones of Table II.
- *Runs B–E* are earlier runs. Their footnotes give what their time and memory measure.
- *Pruning.* Unless stated otherwise, the free sets kept in each copy of the graph are as follows. The between layers keep the 8 free sets that meet a transition set. The approach and depart layers keep those, the free sets that contain the start or the goal, and the free sets of the A*'s plan. Only detours through free sets that touch none of these are lost.
- *Second stage.* The A* rows use the solver-free placement of Section IV-E, without the convex program; "GCS on A* seq." is that convex program.

On this decomposition GCS was about 80 times slower than on the current one (44 s against 0.56 s for one grasp), and its graph was larger: 543 edges against 130 for one grasp. In Runs A–C the full GCS chose the A*'s sets; in Run D its paths were much shorter than the A*'s placement (e.g. 6.30 against 19.13), but that run has no row for GCS on the A* sequence.

TABLE V

ALL A* AND GCS RUNS ON THE EARLIER DECOMPOSITION OF 1CAP_2OBS (42 FREE SETS, $L_{\max} = 0.81$ RAD), GROUPED BY RUN. EACH RUN HAS ITS OWN START AND GOAL. $|V|/|E|$: THE GRAPH EACH METHOD WORKS ON (FOR GCS ON THE A* SEQUENCE, THE SETS OF THAT SEQUENCE). LENGTH: EUCLIDEAN, IN (x, α). "REACH. A* (OURS)" IS THE METHOD OF THIS PAPER. ALL RETURNED PLANS ARE CERTIFIED: EVERY MOTION LIES IN ITS CONVEX SET AND RESPECTS THE COUPLING. MOSEK UNLESS STATED. MACHINE: AMD RYZEN 9 9950X (16 CORES), 60 GB.

Run	D	Method	$\lvert V\rvert/\lvert E\rvert$	Grasps	Length	Time [s]	Memory [GB]	Outcome
A[a]	0.5	Reach. A* (ours)	50 / 900	1	18.06	0.0037	<0.001	solved, 83 nodes expanded
		GCS on A* seq.	8 / 7	1	11.49	0.016	<0.001	solved
		Full GCS	48 / 543	1	11.49	44	4.0	solved, cost 21.50
	1.5	Reach. A* (ours)	50 / 900	2	20.30	0.0048	<0.001	solved, 122 nodes expanded
		GCS on A* seq.	10 / 9	2	13.62	0.022	<0.001	solved
		Full GCS	64 / 749	2	13.62	93	5.6	solved, cost 33.62
	6	Reach. A* (ours)	50 / 900	8	31.16	0.014	<0.001	solved, 369 nodes expanded
		GCS on A* seq.	22 / 21	8	24.48	0.052	0.001	solved
		Full GCS	not run (see Run E)					
B[b]	0.5	Reach. A* (ours)	50 / 900	1	12.07	0.0037	<0.001	solved
		GCS on A* seq.	7 / 6	1	11.98	0.02	<0.001	solved
		Full GCS	61 / 911	1	11.98	243	9.2	solved, cost 21.99
	1.5	Reach. A* (ours)	50 / 900	2	14.29	0.004	<0.001	solved
		GCS on A* seq.	9 / 8	2	14.16	0.023	<0.001	solved
		Full GCS	77 / 1117	2	14.16	421	≥5.7	solved, cost 34.16
	6	Reach. A* (ours)	50 / 900	8	25.15	0.0091	<0.001	solved
		GCS on A* seq.	21 / 20	8	25.02	0.046	<0.001	solved
C[c]	0.5	Reach. A* (ours)	50 / 900	1	15.29	–	–	solved
		GCS on A* seq.	–	1	12.47	0.05	–	solved
		Full GCS	–	1	12.47	231	–	solved
	1.5	Reach. A* (ours)	50 / 900	2	17.56	–	–	solved
		GCS on A* seq.	–	2	14.63	0.01	–	solved
		Full GCS	–	2	14.63	254	–	solved
	6	Reach. A* (ours)	50 / 900	8	28.43	–	–	solved
		GCS on A* seq.	–	8	25.49	0.05	–	solved
D[d]	0.5	Reach. A* (ours)	50 / 900	1	19.13	0.004	–	solved
		Full GCS	63 / 1049	1	6.30	414	≤12.1	solved, cost 16.31 (bound 13.50)
	1.5	Reach. A* (ours)	50 / 900	2	21.36	0.005	–	solved
		Full GCS	79 / 1255	2	8.52	415	≤15.6	solved, cost 28.52 (bound 26.90)
	2.4	Reach. A* (ours)	50 / 900	3	23.49	0.006	–	solved
		Full GCS	95 / 1461	3	10.69	680	≤18.2	solved, cost 40.69 (bound 40.27)
	3.2	Reach. A* (ours)	50 / 900	4	25.43	0.005	–	solved
		Full GCS	111 / 1667	4	12.62	200	≤18.2	solved, cost 52.63 (bound 52.13)
E[e]	6	Reach. A* (ours)	50 / 900	8	26.06	0.013[f]	–	solved, 369 nodes expanded[f]
		GCS on A* seq.	20 / 19	8	25.96	0.018	≤0.36	solved
		Full GCS, pruned	179 / 2280	–	–	140	13.7	relaxation solved (bound 98.66)
		rounding	179 / 2280	–	–	–	>15[g]	stopped by us
		Full GCS, 42 / 8 / 42	≈240 / 5.4k	–	–	≈180	>59[h]	killed by the OS
		Full GCS, all 42	≈540 / 20k	–	–	≈1200	>35[h]	killed by the OS
		Full GCS, all 42, Clarabel[i]	≈540 / 20k	–	–	540[j]	>57, 57, 35[h]	killed by the OS (3 runs)
		1 grasp layer, 42 / – / 42	94 / 3554	–	–	–	>12[g]	memory test; stopped by us

[a] Start, goal and sets saved; reproduces exactly. Times and memory as in the setup: one MOSEK licence held throughout, and each full GCS solve in a fresh process. An earlier measurement of this run solved GCS in forked processes, where MOSEK ran single-threaded and took 191 and 403 s. Run B held a licence too; Runs C–E did not, so their times include a licence checkout per small program. [b] As A, but both full GCS solves ran in one process. The second figure excludes memory the first left allocated, so it understates the need. [c] GCS times are the convex program or `SolvePath` only. Memory and the A* time were not recorded. [d] `SolvePath` only, with one MOSEK licence checkout per solve. Memory: peak of a process that ran the previous turns first, an upper bound. Bound: cost of the convex relaxation, solved separately (415, 416, 674 and 199 s); costs include 10 per grasp. GCS on the A* sequence failed here on a bug in mapping the A* plan, since fixed. [e] Each row its own process and start and goal. Free sets kept in each copy of the graph (approach / between / depart): "pruned" = start or goal sets plus the 8 sets that meet a transition set / those 8 / those 8 plus goal sets. [f] From the run of the "all 42" row. [g] A memory limit we set to protect the machine: the run needed more, not that it cannot be solved. [h] Size of the process when the operating system killed it. It depends on the memory the rest of the machine used at the time: a lower bound. [i] The MOSEK licence path was not set in that shell, so Drake fell back to Clarabel. [j] Third run; the first two were not timed.

Acknowledgment

[TODO: acknowledgments; funding is in the title footnote.]

References

[1] T. Siméon, J.-P. Laumond, J. Cortés, and A. Sahbani, "Manipulation planning with probabilistic roadmaps," *The International Journal of Robotics Research*, vol. 23, no. 7–8, pp. 729–746, 2004.

[2] K. Hauser and J.-C. Latombe, "Multi-modal motion planning in non-expansive spaces," *The International Journal of Robotics Research*, vol. 29, no. 7, pp. 897–915, 2010.

[3] J. Mirabel, S. Tonneau, P. Fernbach *et al.*, "HPP: A new software for constrained motion planning," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 383–389.

[4] R. Deits and R. Tedrake, "Footstep planning on uneven terrain with mixed-integer convex optimization," in *Proc. IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, 2014, pp. 279–286.

[5] M. Toussaint, "Logic-geometric programming: An optimization-based approach to combined task and motion planning," in *Proc. International Joint Conference on Artificial Intelligence (IJCAI)*, 2015, pp. 1930–1936.

[6] S. Levit and M. Toussaint, "Regrasp maps for sequential manipulation planning," *arXiv preprint arXiv:2507.12407*, 2025.

[7] W. Wan and K. Harada, "Regrasp planning using 10,000s of grasps," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 1929–1936.

[8] P. Lertkultanon and Q.-C. Pham, "A certified-complete bimanual manipulation planner," *IEEE Transactions on Automation Science and Engineering*, vol. 15, no. 3, pp. 1355–1368, 2018.

[9] R. Deits and R. Tedrake, "Computing large convex regions of obstacle-free space through semidefinite programming," in *Algorithmic Foundations of Robotics XI*. Springer, 2015, pp. 109–124.

[10] H. Dai, A. Amice, P. Werner *et al.*, "Certified polyhedral decompositions of collision-free configuration space," *The International Journal of Robotics Research*, vol. 43, no. 9, pp. 1322–1341, 2024.

[11] M. Petersen and R. Tedrake, "Growing convex collision-free regions in configuration space using nonlinear programming," *arXiv preprint arXiv:2303.14737*, 2023.

[12] P. Werner, A. Amice, T. Marcucci *et al.*, "Approximating robot configuration spaces with few convex sets using clique covers of visibility graphs," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[13] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, "Motion planning around obstacles with convex optimization," *Science Robotics*, vol. 8, no. 84, p. eadf7843, 2023.

[14] J. Wang and S. Tonneau, "CASSR: Continuous A-star search through reachability for real time footstep planning," *arXiv preprint arXiv:2603.02989*, 2026.

[15] P. Tournassoud, T. Lozano-Pérez, and E. Mazer, "Regrasping," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 1987, pp. 1924–1928.

[16] R. Alami, T. Siméon, and J.-P. Laumond, "A geometrical approach to planning manipulation tasks. the case of discrete placements and grasps," in *The Fifth International Symposium on Robotics Research*. MIT Press, 1990, pp. 453–463.

[17] P. S. Schmitt, W. Neubauer, W. Feiten *et al.*, "Optimal, sampling-based manipulation planning," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2017.

[18] Z. Kingston, M. Moll, and L. E. Kavraki, "Sampling-based methods for motion planning with constraints," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 1, pp. 159–185, 2018.

[19] C. R. Garrett, R. Chitnis, R. Holladay *et al.*, "Integrated task and motion planning," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, pp. 265–293, 2021.

[20] S. Liu, T. Zhao, B. P. Graesdal *et al.*, "Approximately optimal global planning for contact-rich SE(2) manipulation on a graph of reachable sets," *arXiv preprint arXiv:2601.10827*, 2026.

[21] R. J. Griffin, G. Wiedebach, S. McCrory *et al.*, "Footstep planning for autonomous walking over rough terrain," in *Proc. IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, 2019, pp. 9–16.

[22] S. Tonneau, D. Song, P. Fernbach *et al.*, "SL1M: Sparse L1-norm minimization for contact planning on uneven terrain," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 6604–6610.

[23] T. Marcucci, J. Umenberger, P. Parrilo, and R. Tedrake, "Shortest paths in graphs of convex sets," *SIAM Journal on Optimization*, vol. 34, no. 1, pp. 507–532, 2024.

[24] B. P. Graesdal, S. Y. C. Chia, T. Marcucci *et al.*, "Towards tight convex relaxations for contact-rich manipulation," in *Proc. Robotics: Science and Systems (RSS)*, 2024.

[25] S. Y. C. Chia, R. H. Jiang, B. P. Graesdal *et al.*, "GCS*: Forward heuristic search on implicit graphs of convex sets," in *Workshop on the Algorithmic Foundations of Robotics (WAFR)*, 2024.

[26] P. Werner, T. Cohn, R. H. Jiang *et al.*, "Faster algorithms for growing collision-free convex polytopes in robot configuration space," in *International Symposium on Robotics Research (ISRR)*, 2024.

[27] P. Werner, R. Cheng, T. Stewart *et al.*, "Superfast configuration-space convex set computation on GPUs for online motion planning," *arXiv preprint arXiv:2504.10783*, 2025.

[28] R. Tedrake and the Drake Development Team, "Drake: Model-based design and verification for robotics," https://drake.mit.edu, 2019.

[29] M. Toussaint, C. V. Braun, and J. Ortiz-Haro, "NLP sampling: Combining MCMC and NLP methods for diverse constrained sampling," *arXiv preprint arXiv:2407.03035*, 2024.

[30] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[31] J. P. Vielma, "Mixed integer linear programming formulation techniques," *SIAM Review*, vol. 57, no. 1, pp. 3–57, 2015.

[32] S. Haddadin, S. Parusel, L. Johannsmeier *et al.*, "The Franka Emika robot: A reference platform for robotics research and education," *IEEE Robotics & Automation Magazine*, vol. 29, no. 2, pp. 46–64, 2022.

[33] D. Goldfarb and A. Idnani, "A numerically stable dual method for solving strictly convex quadratic programs," *Mathematical Programming*, vol. 27, no. 1, pp. 1–33, 1983.

[34] MOSEK ApS, "The MOSEK optimization toolbox," https://www.mosek.com, 2024.